

Refertium Portal: Branch Changes and Recognition Testing Checklist

Branch: andrey-design-fixes

Recognition Testing Configuration

To test speech recognition through the Cloudflare proxy, the local backend, Cloudflare Worker, PostgreSQL, and user license state must all be aligned.

Backend Environment

Set these values for the portal backend:

```
REFERTIUM_PROXY_MODE=cloudflare
CLOUDFLARE_PROXY_URL=https://<your-worker>.<account>.workers.dev
CLOUDFLARE_PROXY_AUTH=<same value as Worker PROXY_AUTH_TOKEN>
WORKER_SHARED_SECRET=<same value as Worker WORKER_SHARED_SECRET>
PUBLIC_BASE_URL=http://localhost:47825
DATABASE_URL=postgres://<user>:<password>@localhost:55432/<database>
PGSSLMODE=disable
```

For direct local OpenAI testing without Cloudflare, use:

```
REFERTIUM_PROXY_MODE=local
OPENAI_API_KEY=<real OpenAI API key>
```

Cloudflare Worker Secrets

Set these secrets in the Worker:

```
PROXY_AUTH_TOKEN=<same value as backend CLOUDFLARE_PROXY_AUTH>
WORKER_SHARED_SECRET=<same value as backend WORKER_SHARED_SECRET>
REFERTIUM_DASHBOARD_URL=https://<public-backend-url>
OPENAI_API_KEY=<for /v1/audio/transcriptions and AI calls>
DEEPGRAM_API_KEY=<for /v1/deepgram/token if Deepgram recognition is tested>
```

Cloudflare cannot call `http://localhost:47825` directly. For a full local test through Cloudflare, expose the local backend with a public tunnel such as `cloudflared tunnel` or `ngrok`, then use that public URL as `REFERTIUM_DASHBOARD_URL`.

Runtime Requirements

- PostgreSQL must be running and reachable by the backend.
- The test user must have an active license.
- The license must have available token and dictation limits.
- The user must open the app through the portal, so the backend can issue the active app session.
- Old app tabs are intentionally invalidated when a new app session is opened.
- The browser must allow microphone access.
- The Worker now accepts both `X-Auth-Token` and `Authorization: Bearer <token>` for proxy authentication, matching the current app templates.

Completed Work in the Branch

Architecture

- Extracted security helpers from `server.js` into `backend/lib/security.js`.
- Extracted payment validation and Stripe Checkout parameter helpers into `backend/lib/payments.js`.
- Kept the changes scoped to the existing native Node.js HTTP architecture.
- Added a focused test structure under `backend/test`.
- Updated Cloudflare Worker authentication compatibility for current premium templates.

Database

- Added PostgreSQL support through the `pg` package.
- Added Docker Compose configuration for a local PostgreSQL instance.
- Moved users, licenses, sessions, login attempts, payment links, and payments to PostgreSQL.
- Added one-time migration from `backend/data/db.json` when `settings[key=import]` is not equal to `1`.
- Stored the import completion flag in the `settings` table.
- Kept licenses separate from sessions: sessions represent authentication/app access, while licenses represent recognition usage rights and limits.
- Added key indexes for users, sessions, licenses, login attempts, payment links, and payments.

Security

- Added secure cookie defaults: `HttpOnly`, `Secure`, and `SameSite`.
- Persisted sessions in PostgreSQL so users are not logged out by an ordinary server restart.
- Added login rate limiting: after 5 failed attempts, login returns `429` and unlocks after one hour.
- Added single active app session enforcement, so newly opened app sessions invalidate previous app tabs.
- Blocked normal users from opening another user's `/app/:userId`.
- Verified normal users cannot access admin and finance API data.
- Added Worker-to-dashboard authorization using `proxyToken`, `appSession`, and `WORKER_SHARED_SECRET`.
- Removed the Worker development shared-secret fallback for production-sensitive callbacks.
- Hardened proxy usage reporting so recognition and AI usage is checked against the database-backed license state.

Payments

- Added Stripe Checkout support for arbitrary payment amounts.
- Avoided requiring predefined Stripe Products or Prices.
- Added admin payment link generation.
- Added admin UI action to send a payment link to a user.
- Added Stripe webhook handling for completed checkout sessions.
- Stored payment links and payments in PostgreSQL.
- Mapped each successful payment back to the relevant user through Stripe metadata and client reference data.

Tests and Verification

- Added unit tests for payment month keys, payment amount validation, Stripe amount conversion, signed payment URLs, Stripe Checkout metadata, secure cookies, cookie clearing, cookie parsing, password verification, and login rate-limit key normalization.
- Verified `npm test` passes with 11 tests.
- Verified `backend/server.js`, `backend/lib/security.js`, and `backend/lib/payments.js` with `node --check`.
- Verified the Cloudflare Worker syntax with `node --check`.
- Smoke-tested local backend startup on `http://localhost:47825/`.
- Smoke-tested local PostgreSQL-backed credentials: `admin/admin` and `user/user`.
- Verified that `user/user` can open its own app when the license is active.
- Verified that `user/user` cannot access admin-only endpoints.

Remaining Configuration Needed for Recognition Testing

- Public Worker URL.
- Worker `PROXY_AUTH_TOKEN`.
- Shared Worker/backend `WORKER_SHARED_SECRET`.
- Public backend tunnel URL for `REFERTIUM_DASHBOARD_URL`.
- Provider API key: `OPENAI_API_KEY`.
- Optional provider API key: `DEEPGRAM_API_KEY`.
- Active PostgreSQL-backed user license with available limits.